

CSE 4621: Machine Learning Language Modeling

Lecture 9

Ishmam Tashdeed
Lecturer, CSE IUT

Lecture Outline

- Language as Data
 - Representing text computationally
 - Language modelling task
- Word Embeddings
 - One-hot encoding and its failures
 - Word2Vec
 - Distributional semantics
- RNNs
 - Architecture and hidden state
 - Backprop through time
 - Vanishing gradients
 - LSTMs and GRUs
- Attention
 - The bottleneck problem
 - Self-attention
 - Multi-head attention
- Transformers
 - Attention Is All You Need
 - Positional encoding
 - Encoder-decoder structure
 - Transformers in other fields

Language as data

- Images have fixed $H \times W \times C$ shape. Sentences have variable lengths. We can't just flatten a sentence into a vector of fixed size.
- The meaning of a word depends on the context.

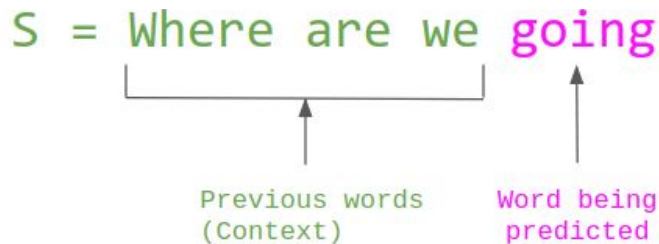
“I never said Messi cheated.”

- Convolutional sliding windows (fixed receptive field) can't capture arbitrarily long-range dependencies.

Language as data

Given a sequence of words $w_1, w_2, \dots, w_{t-1}$ predict the next word w_t : $P(w_t | w_1, w_2, \dots, w_{t-1})$

- "The cat sat on the ____" $\rightarrow$ *mat* (high probability), *quantum* (near zero)
- "The capital of Bangladesh is ____" $\rightarrow$ *Dhaka*
- "2 + 2 = ____" $\rightarrow$ 4



$$P(S) = P(\text{Where}) \times P(\text{are} | \text{Where}) \times P(\text{we} | \text{Where are}) \times P(\text{going} | \text{Where are we})$$

Representing Words

Vocabulary of size V ; word w_i is represented as a vector of all zeros except a single 1 at position i

Some issues arise from this:

- Dimensionality $\rightarrow$ large weights
- No similarity $\rightarrow$ cosine similarity = 0
- No generalization $\rightarrow$ all words are independent of each other

One-Hot Encoding: 5 Vocabulary Words $\times$ 5 Dimensions

Each word is represented by a vector with exactly one 1 and the rest 0s.

Vocabulary (word)	One-Hot Dimensions				
	1	2	3	4	5
the	1	0	0	0	0
cat	0	1	0	0	0
sat	0	0	1	0	0
on	0	0	0	1	0
mat	0	0	0	0	1

Extremely sparse: only 1 non-zero entry per row
(80% zeros)

Dot Products Between Different Words

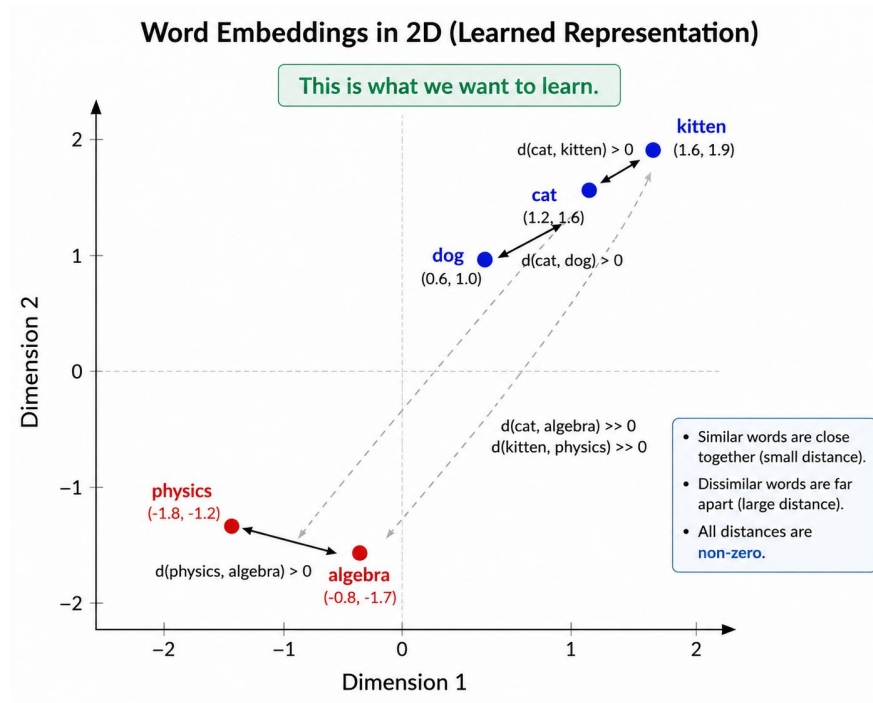
the $\cdot$ cat = 0
the $\cdot$ sat = 0
the $\cdot$ on = 0
the $\cdot$ mat = 0
:
cat $\cdot$ sat = 0
:
on $\cdot$ mat = 0

All different words are **orthogonal** (dot product = 0).
There is no inherent similarity between one-hot vectors.

Representing Words

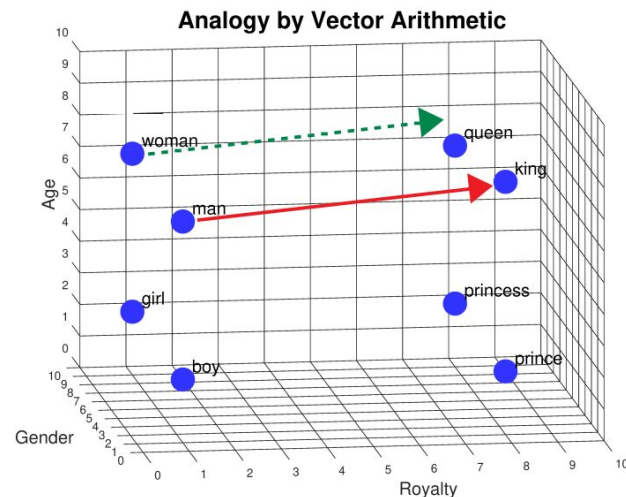
Replace each word with a dense,
low-dimensional, continuous vector learned
from data

But how?



Representing Words

- At the beginning, every word gets a random vector
 - cat = [0.13, -0.44, 0.91, ...]
 - dog = [-0.72, 0.08, 0.35, ...]
 - pizza = [0.41, 0.66, -0.12, ...]
- The model sees many training examples from text:
 - The cat drinks milk
- The model is forced to make predictions on:
 - given the center word, predict surrounding words (Skipgram)
 - given the surrounding words, predict center word (CBOW)
- To become good at prediction, the model is forced to invent useful internal word representations



Representing Words

“You shall know a word by the company it keeps ” — Firth, 1957

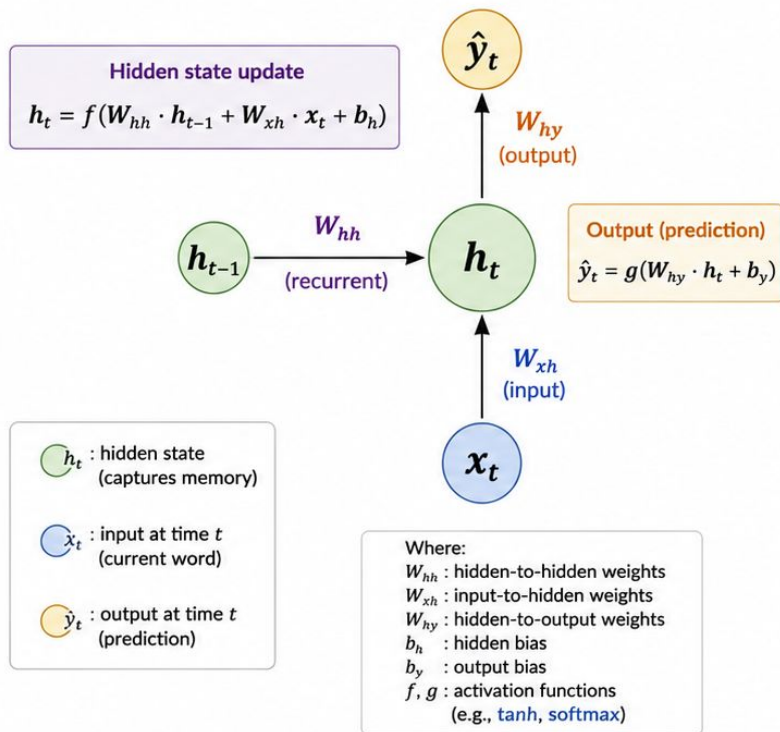
- Words that appear in similar contexts have similar meanings
- Gradient descent on word prediction automatically encodes this
- Co-occurring words pull their embeddings toward each other during training

The style of training is called self-supervised learning → Creating labeled data from completely unlabeled data

Recurrent Neural Networks (RNNs)

1) Single time step (one RNN node)

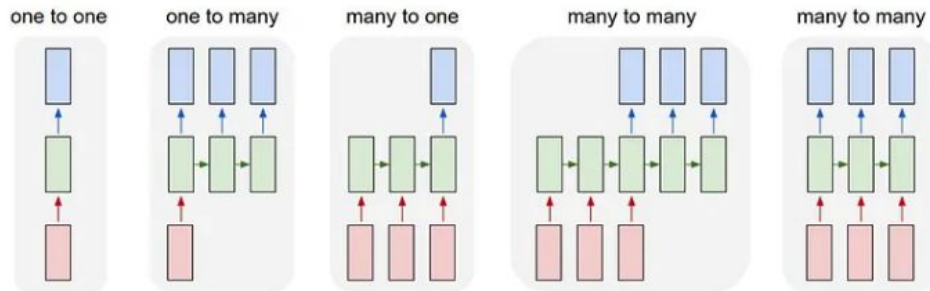
Internal calculations



Giving networks sequential memory $\rightarrow$ maintain a hidden state h_t as a running summary

One set of weights are shared across all T steps

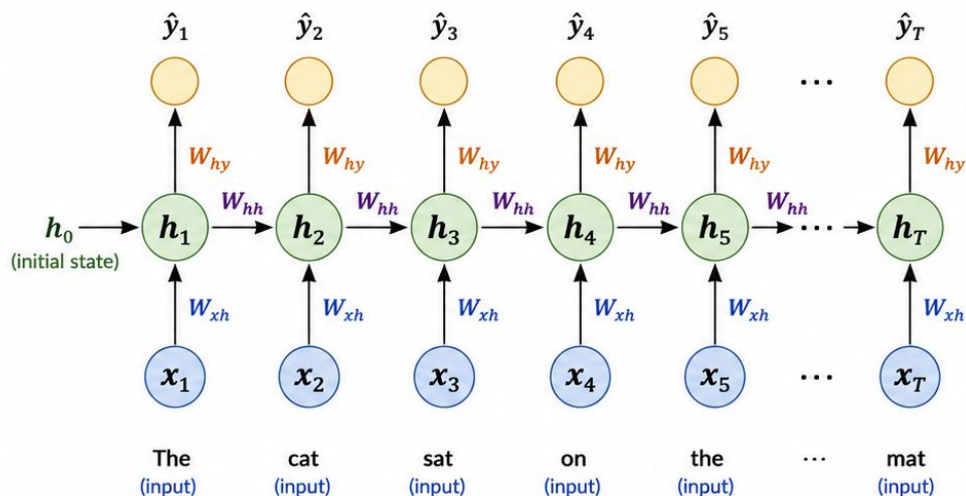
Similar to CNN models sharing filter weights across the image, but for RNNs, it is in temporal dimension



Recurrent Neural Networks (RNNs)

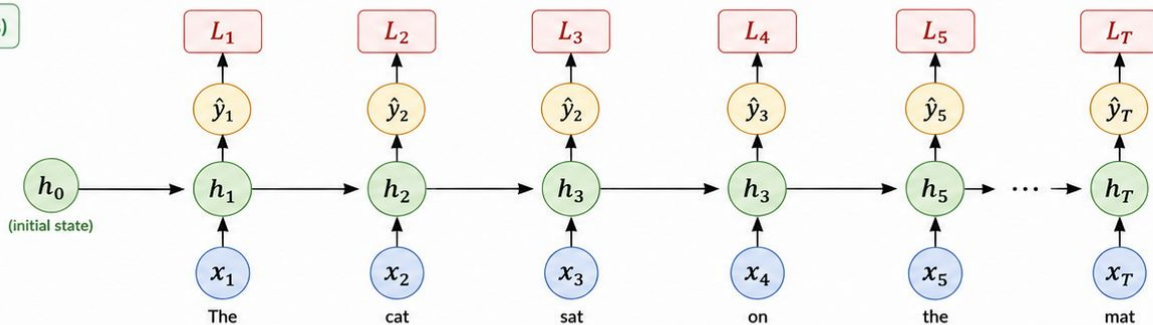
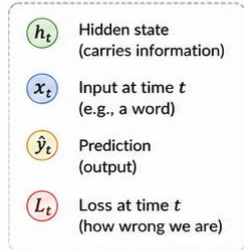
2) RNN unfolded across time (sequence of words)

The same node is repeated over time, sharing parameters



Vanishing and Exploding Gradients

1) Forward Pass (making predictions)



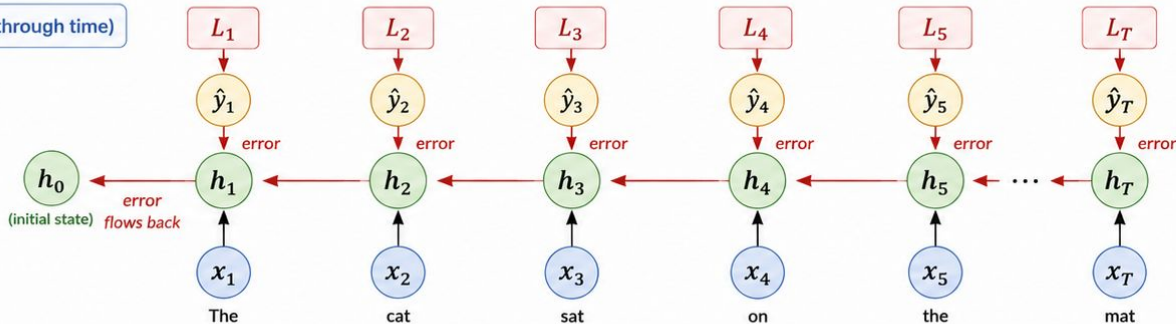
We get a loss at every time step comparing prediction with the true next word.

2) Backward Pass (backpropagation through time)

Start from the last time step and move backward.

The error at later steps flows back through the hidden states to earlier steps.

This lets the model learn from the consequences of its earlier decisions.



Errors (gradients) flow backward through time via the hidden states (backpropagation through time).



BPTT “unrolls” the RNN over time, then applies regular backpropagation on this unrolled chain to update the shared parameters.

Vanishing and Exploding Gradients

This sequential processing and backpropagation often gives rise to:

- Vanishing Gradients:
 - Gradients shrink exponentially
 - Early layers learn nothing
 - RNN forgets after 10~20 steps
- Exploding Gradients:
 - Gradients grow exponentially
 - NaN parameters
 - Training diverges

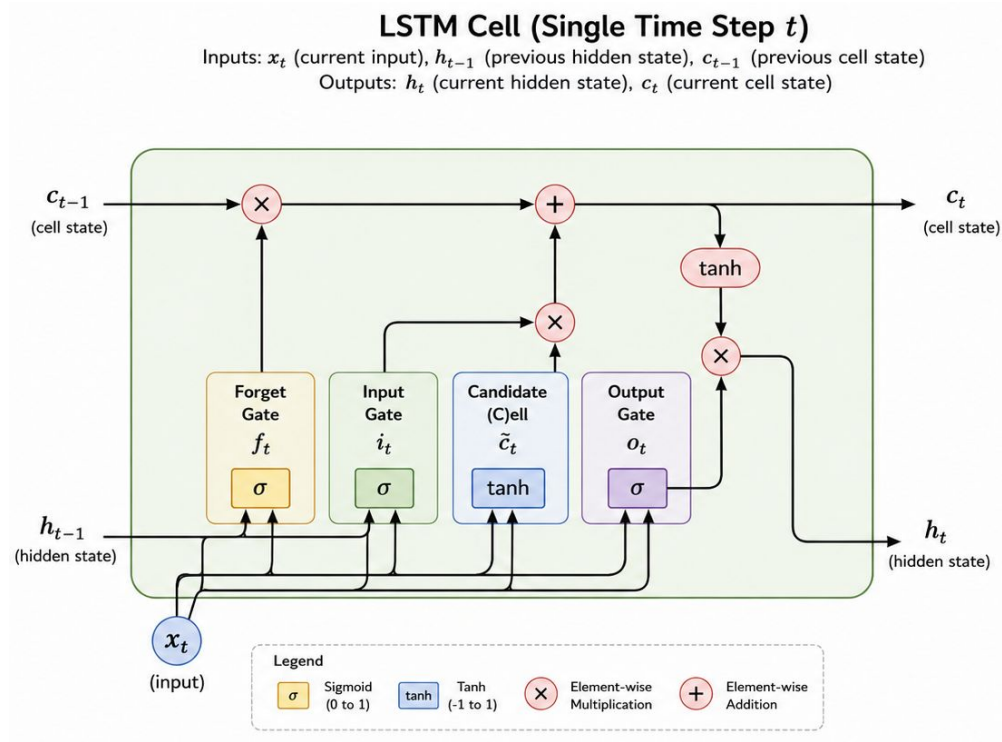
Thus, RNNs can not feasibly remember words that are more than ~20 steps away!

Long Short-Term Memory (LSTM)

Essentially has 3 gates:

- Input gate
- Forget gate
- Output gate

An additional cell state c_t acts as a conveyor belt of information

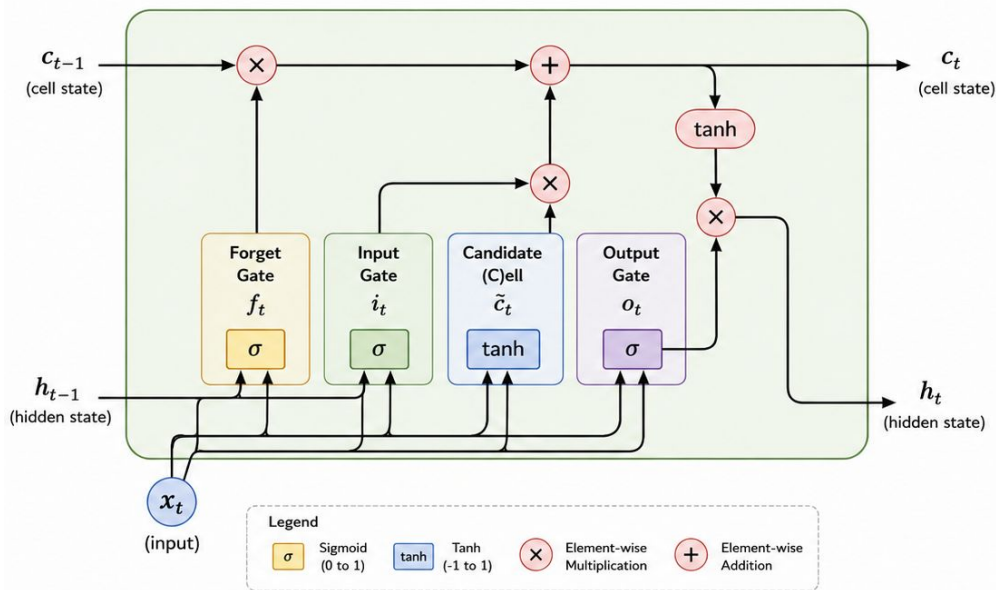


Long Short-Term Memory (LSTM)

LSTM Cell (Single Time Step t)

Inputs: x_t (current input), h_{t-1} (previous hidden state), c_{t-1} (previous cell state)

Outputs: h_t (current hidden state), c_t (current cell state)



Internal Calculations

1. **Forget gate** (decide what to forget from cell state)

$$f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f)$$

2. **Input gate** (decide what new information to write)

$$i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i)$$

3. **Candidate cell** (create new candidate values)

$$\tilde{c}_t = \tanh(W_c \cdot [h_{t-1}, x_t] + b_c)$$

4. **Update cell state** (combine old state and new candidate)

$$c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t$$

5. **Output gate** (decide what to output)

$$o_t = \sigma(W_o \cdot [h_{t-1}, x_t] + b_o)$$

6. **Hidden state** (compute new hidden state)

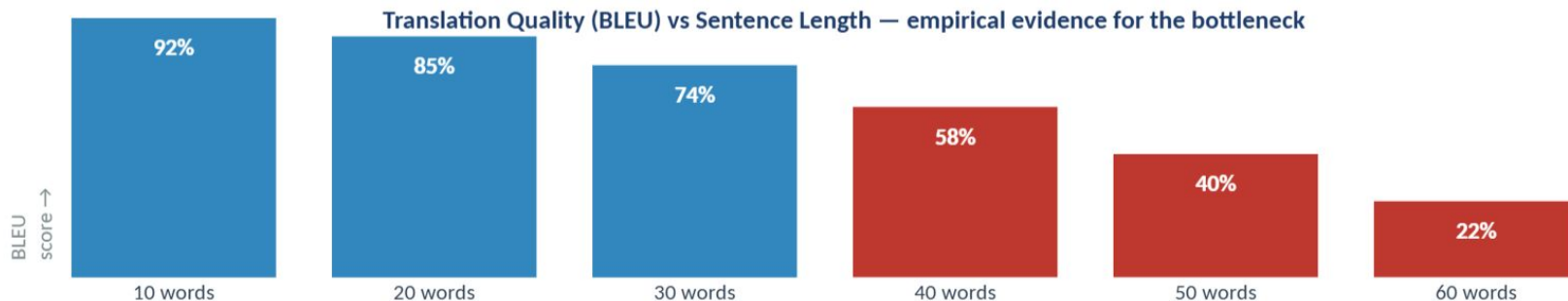
$$h_t = o_t \odot \tanh(c_t)$$

Where:

- $[,]$: Concatenation
- W_f, W_i, W_c, W_o : Weight matrices
- b_f, b_i, b_c, b_o : Bias vectors
- $\odot$: Element-wise multiplication
- σ : Sigmoid function
- $\tanh$: Hyperbolic tangent function

Sequence-to-Sequence Bottleneck Problem

The encoder-decoder compresses the entire input sequence into one fixed size context vector

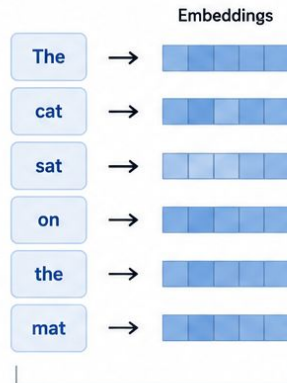


📉 Quality degrades dramatically for long sentences — the context vector loses information

Attention Mechanism

1 Input sequence

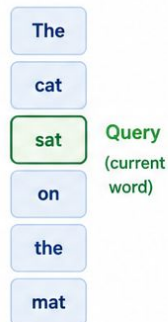
A sequence of words is converted to embeddings.



These embeddings are the input to the attention layer.

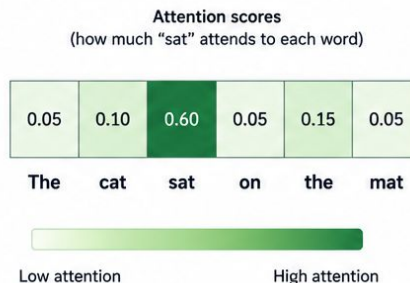
2 For a target word...

We want to understand the word "sat", so we use it as the query.



3 Compute attention scores

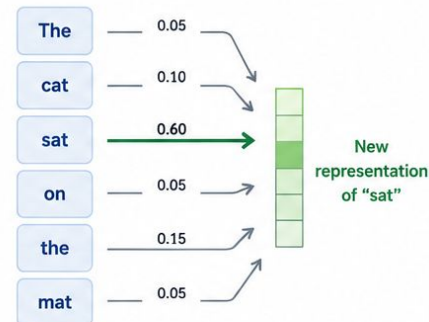
Compare the query with all words (keys) to get relevance scores.



The model pays the most attention to words that are most useful for understanding "sat".

4 Compute the output

Take a weighted sum of all word embeddings using the scores.



The output is a new representation of "sat" that focuses on the most relevant words (here: "cat", "the").

In short



Attention computes how relevant every other word is to the current word, then builds a representation by focusing more on the important ones and less on the others.

[Blue box] Input / Embedding

[Light green box] Scores / Weights

[Dark green box] Output Representation

→ Data flow

→ Stronger focus (higher weight)

— Weaker focus (lower weight)

Textbook Chapters

- [Goodfellow] - Chapter 9
- [Alpaydin] - Chapter 12

Recommended Resources

1. [Meme search using pretrained word2vec](#)
2. [Recurrent Neural Networks \(RNNs\)](#)

Thank you